Replication: Estimating Marketing Component Effects

Double Machine Learning from Targeted Digital Promotions (Ellickson, Kar, and Reeder, Marketing Science 2023)

Weiheng Zhang

September 05, 2026

Contents

1 The paper in one page

The question is which parts of a promotional email cause customers to open it, not whether the email as a whole works. Ellickson, Kar, and Reeder (2023) observe 34 email promotions sent by an apparel retailer to 1.3 million customers over two months. Each email is a bundle of components: a merchandise category, a price or nonprice promotion, semantic cues in the subject line, and its character length. Outcomes are observed at three stages of the funnel: open, purchase, and purchase amount.

Targeting is the identification problem, and the firm's own targeting variables are the solution. The firm chose whom to send each email using fifteen recency, frequency, monetary, and demographic variables that the authors observe. Assignment is therefore unconfounded given those variables (their Assumption 1), and the residual randomization in the firm's rule gives overlap (Assumption 2). The paper spends Section 5 on checking overlap and balance because everything that follows rests on those two assumptions.

The method has two stages, and both consume one object: the doubly robust score. In the first stage, for each email j against a baseline email, they build the score

$$\psi_i = \hat{\mu}_1(x_i) - \hat{\mu}_0(x_i) + \frac{W_i}{\hat{e}_t(x_i)}\,(y_i - \hat{\mu}_1(x_i)) - \frac{1 - W_i}{\hat{e}_c(x_i)}\,(y_i - \hat{\mu}_0(x_i)),$$

with random forests for the outcome models μ and the propensities e, trained on one half of the sample and evaluated on the other half. The mean of ψ_i is the pairwise ATE (Figures 1 to 3 of the paper). In the second stage they regress the pooled scores on the component differences between each email and the baseline, following Semenova and Chernozhukov (2021); each coefficient is the marginal effect of one component (Table 10), and the same regression by customer segment gives heterogeneous component effects (Tables 12 and 13).

Why two stages rather than one regression of opens on components. A regression of outcomes on components confounds the component effects with targeting, because the firm sends different bundles to different customers. The score removes targeting once, using all fifteen covariates and flexible learners, and leaves a variable whose conditional mean is the treatment effect. The projection then answers the decomposition question with plain least squares and robust standard errors, which is why the second stage can be simple.

The public replication package cannot reproduce the paper's numbers, and says so. The real data are under a nondisclosure agreement. The package holds a synthetic dataset with the same variable types, the authors' two R scripts, and the authors' own first-stage output on the synthetic data. Table 1 lists them. This note replicates the workflow on those files, matches our package's estimates against the authors' scripts step by step, and keeps the paper's table and figure layouts so that each object can be read next to its published counterpart.

Table 1: Files in the replication package and their role in this note.

File	Content	Used for
DigitalPromo_Replication.csv	221,188 customers, one row each; 6 emails (email 6 is the baseline); outcome Opened; 4 covariates	descriptives, balance, first stage, segments
Step2_Analysis_Mat.csv	the authors' first-stage output: 251,804 scores for emails 1–5 against the baseline	benchmark for our first stage
EmailComponents.csv	component differences (email minus baseline) for emails 1–5: Characters, Component1–3	second stage
FirstStage_ScoreCreation_Replication.r	score construction: ranger forests, 1,000 trees, seed 227	benchmark rerun
SecondStage_Projection_Replication.r	trimming, stratified subsample of 25,000 rows per pair, lm with HC1	benchmark rerun

What is lost relative to the paper. Only the open stage exists, so every three-outcome table has one

outcome column. There are five treatment emails instead of 33, four covariates instead of fifteen, four generic components instead of eighteen named ones, and no customer receives more than one email. The segment variable shipped with the data is constant, so segments are rebuilt from the covariates as the paper does.

2 Data

2.1 Emails and outcomes

The paper's Table 2 reports raw open rates by component to show that components differ in performance before any adjustment. The table exists to motivate the decomposition, and its last column (purchase amount conditional on purchase) previews that top-of-funnel and bottom-of-funnel rankings disagree. The authors are careful to call these associations: the customers exposed to each component were chosen by the firm.

```
promo <- read.csv(file.path(data_path, "DigitalPromo_Replication.csv"))
components <- read.csv(file.path(data_path, "EmailComponents.csv"))
step2_authors <- read.csv(file.path(data_path, "Step2_Analysis_Mat.csv"))
promo$treated_email <- promo$PromoID != baseline_email
c(customers = nrow(promo), emails = length(unique(promo$PromoID)),
  customers_with_more_than_one_email = sum(table(promo$ID) > 1))
#>                          customers                             emails
#>                             221188                                  6
#> customers_with_more_than_one_email
#>                                  0
```

Table 2: Emails in the replication data: recipients, raw open rate, and component differences from the baseline email (the analogue of the paper's Tables 1 and 2). Component values are differences from email 6, so the baseline row is empty.

Email	Role	Number of customers	Open rate	Characters	Component 1	Component 2	Component 3
1	treatment	12,525	0.282	-8	0	0	0
2	treatment	28,091	0.203	-7	0	1	-0.2
3	treatment	71,678	0.100	-1	1	0	-0.4
4	treatment	12,649	0.245	-10	0	0	0.1
5	treatment	47,153	0.089	-11	1	0	-0.4
6	baseline	49,092	0.092	–	–	–	–

In the synthetic data the raw open rates already spread from 0.089 to 0.282. Whether that spread reflects the emails or the customers they were sent to is exactly what the first stage has to settle.

2.2 Pretreatment covariates

The paper's Table 3 lists the fifteen targeting variables with their means and standard deviations to make one point: these are the firm's own targeting inputs, so conditioning on them is conditioning on the assignment rule. The standard deviations of the RFM variables are large relative to their means, which the authors read as evidence of active segmentation, and the demographic variables barely vary across emails, which suggests they play little role in targeting.

Why the layout groups variables into recency, frequency, monetary, and demographic families. The families map onto the firm's targeting logic, and the same grouping reappears in every later table (balance, the two emails compared in Table 8, the segments in Table 11), so a reader can follow one variable through the paper.

Table 3: Pretreatment covariates in the replication data (the analogue of the paper's Table 3). The synthetic data keep one variable per family of the firm's fifteen targeting variables.

Recipient characteristic	Variable	Description	Mean	Standard deviation
Demographic	Demographic	Demographic bracket of the recipient (categorical, 1–3)	2.29	0.54
Monetary	Monetary	Monetary value of past purchases (dollars)	43.84	47.10
Frequency	Frequency	Number of past purchases	3.05	4.93
Recency	Recency	Days since the recipient's last action	587.80	743.68

3 Overlap and balance

3.1 Why the paper checks balance before estimating anything

Unconfoundedness holds by construction here, but overlap does not, so the authors test overlap and balance rather than assume them. A firm that targets algorithmically could target deterministically: if a rule sends an email only to customers with a birthday this month, no other customer has a positive propensity and the effect for them is not identified. Section 5 of the paper therefore does three things: it estimates a propensity for every email against the pooled population, checks that the propensities are not degenerate, and reports population standardized biases (PSB) for every covariate and email.

The PSB of McCaffrey et al. (2013) is the balance diagnostic for many treatments. For covariate k and email t,

$$\mathrm{PSB}_{tk} = \frac{|\bar{X}_{kt} - \bar{X}_{kp}|}{\sigma_{kp}}, \qquad \bar{X}_{kt} = \frac{\sum_i T_i[t]\, X_{ki}/\hat{e}_t(X_i)}{\sum_i T_i[t]/\hat{e}_t(X_i)},$$

where $\hat{e}_t$ is the propensity of email t in the pooled population, and $\bar{X}_{kp}$, σ_{kp} are the pooled mean and standard deviation. It compares the propensity-weighted recipients of each email with the whole population, which is the population the pairwise ATEs refer to. The authors adopt the thresholds 0.20 (small), 0.40 (moderate), and 0.60 (large) and flag anything above 0.25 for a closer look.

3.2 Propensities for each email against the pooled population

We estimate the six pooled propensities as the authors do, with probability forests, and cross-fit them so the weights are honest. The authors' script trains each forest on half of the rows and predicts on the other half. Two-fold cross-fitting is the same idea applied to both halves, so every customer gets an out-of-sample propensity.

```
crossfit_prob <- function(data, target, features, n_trees, seed) {
  set.seed(seed)
  fold <- sample(rep(1:2, length.out = nrow(data)))
  out <- numeric(nrow(data))
  for (k in 1:2) {
    fit <- ranger(x = data[fold != k, features], y = factor(data[[target]][fold != k]),
                  num.trees = n_trees, probability = TRUE, num.threads = n_threads, seed = seed)
    out[fold == k] <- predict(fit, data = data[fold == k, features], num.threads = n_threads)$predictions[, "1"]
  }
  out
}
pooled_ps <- sapply(sort(unique(promo$PromoID)), function(j) {
  promo$W <- as.integer(promo$PromoID == j)
  crossfit_prob(promo, "W", covs, n_trees, seed = 100 + j)
})
colnames(pooled_ps) <- paste0("email", sort(unique(promo$PromoID)))
```

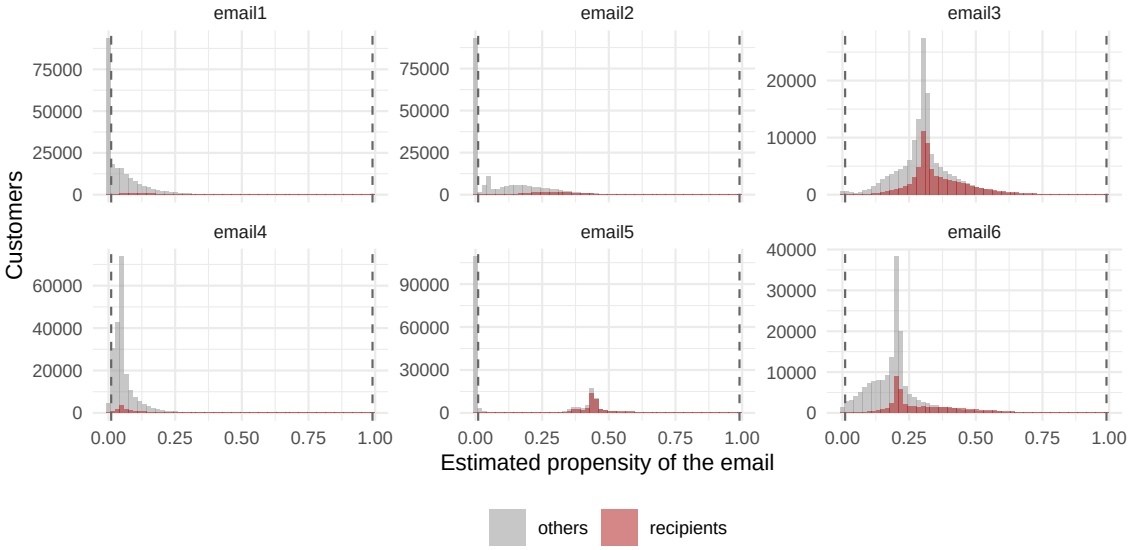

Figure 1: Cross-fitted propensity of each email in the pooled population, recipients of the email against everyone else. The dashed lines are the 0.01 and 0.99 trimming points used in the first stage. Distributions that overlap without mass at the boundaries are the paper's evidence of residual randomization.

3.3 Weighted means and standardized biases

Table 4 reproduces the layout of the paper's Table 4: one row per email, one column per covariate, weighted means in the cells, the pooled mean and standard deviation at the bottom, and a marker on cells whose PSB exceeds 0.20 or 0.40. The layout lets the reader scan a column to see which covariate is targeted and a row to see which email is targeted; the bottom rows give the scale against which the differences are judged.

```
psb_table <- function(data, ps, covs, clip = c(0.01, 0.99)) {
  ps <- pmin(pmax(ps, clip[1]), clip[2])
  pooled_mean <- colMeans(data[, covs])
  pooled_sd <- apply(data[, covs], 2, sd)
  emails <- sort(unique(data$PromoID))
  rows <- lapply(emails, function(j) {
    w <- as.numeric(data$PromoID == j) / ps[, paste0("email", j)]
    wm <- apply(data[, covs], 2, function(x) sum(w * x) / sum(w))
    tibble(PromoID = j, Variable = covs, weighted_mean = wm,
           psb = abs(wm - pooled_mean[covs]) / pooled_sd[covs])
  })
  list(table = bind_rows(rows), pooled_mean = pooled_mean, pooled_sd = pooled_sd)
}
bal <- psb_table(promo, pooled_ps, covs)
```

Reading the tables the way the authors read theirs. Before weighting, the raw means differ sharply across emails: the largest unweighted standardized difference is 0.64 (email 5, Recency), so the synthetic firm targets on recency and frequency as the real one did. After weighting, of the 24 email-by-covariate comparisons, 9 exceed 0.20 and 5 exceed 0.40; the largest is 0.56 (email 1, Recency). The paper found 25 of 510 above 0.20 and one at 0.59, an email personalized to the recipient's birth month, and argued that even that case does not threaten identification because only about 80 percent of eligible customers received it, so the rule was not deterministic. Propensities are clipped at 0.01 and 0.99 before weighting, the paper's trimming points; among recipients of each email the share outside those bounds is at most 0.6 percent, so the weights never explode and the balance the weights deliver is the balance the first stage will rely on.

Why the paper reports maxima in Tables 5 and 6 rather than only the full matrix. With 34 emails and 15 covariates the full matrix has 510 cells; two one-column tables tell the reader at once which emails and which covariates carry the imbalance, and the full matrix is moved to the online appendix. With

Table 4: Propensity-weighted means of the pretreatment covariates by email, with the unweighted pooled mean and standard deviation (the layout of the paper's Table 4). [a]: PSB above 0.20; [b]: PSB above 0.40.

Email	Demographic	Monetary	Frequency	Recency
1	2.30	59.94[a]	4.65[a]	169.74[b]
2	2.31	58.10[a]	4.55[a]	191.56[b]
3	2.29	44.03	3.11	585.99
4	2.28	45.34	2.75	615.34
5	2.28	24.48[b]	0.71[b]	945.98[b]
6	2.29	40.94	2.55	668.73
Pop. mean	2.29	43.84	3.05	587.80
Pop. standard deviation	0.54	47.10	4.93	743.68

Table 5: Maximum PSB across covariates for each email (the layout of the paper's Table 5).

Email	Maximum PSB
1	0.56
2	0.53
3	0.01
4	0.06
5	0.48
6	0.11

Table 6: Maximum PSB across emails for each covariate (the layout of the paper's Table 6).

Recipient characteristic	Pretreatment covariate	Maximum PSB
Demographic	Demographic	0.04
Frequency	Frequency	0.47
Monetary	Monetary	0.41
Recency	Recency	0.56

six emails and four covariates the matrix itself fits, so both views are shown here.

3.4 What the balance check found in the synthetic data

Three emails stay imbalanced after weighting, and the reason is a structural failure of overlap, not a bad propensity model. Recipients of email 1 have recency of at most 403 days while 36 percent of the population is above 500; recipients of email 5 have at most 30 past purchases. No reweighting of the recipients can represent customers who were never sent the email, so the weighted means in Table 4 stay far from the pooled means. This is the failure the paper's footnote 14 calls structural, and the paper's data passed the check; the synthetic data do not for emails 1, 2, and 5. The consequence for the first stage is that the pairwise ATEs for those emails are identified only on the overlapping part of the population. The paper's trimming at propensities of 0.01 and 0.99 makes that restriction explicit, and we keep it.

4 First stage: the doubly robust score

4.1 What the authors' code computes

The score has three ingredients, and each is estimated on a sample that excludes the observation it scores. For treatment email j and the baseline b, the authors' script (a) splits the baseline recipients

in half and the recipients of j one quarter to three quarters, training on the first parts and scoring the second; (b) fits a probability forest for "received j" against every other row of the data, and another for "received b" against every other row, which are the pooled propensities of Section 3 rather than the propensity of j within the pair; (c) fits one probability forest of the outcome on each arm. The propensities are scaled by the ratio of customers to rows, the stabilized weight of Hernán and Robins (2006), which is one here because no customer appears twice.

The score written in the code differs from the formula in the paper in one place. The paper divides the control residual by the control propensity $e_c(x)$; the script divides both residuals by the treatment propensity $e_t(x)$. The check below recomputes the shipped scores from the shipped propensities and outcome predictions with the code's formula and reproduces them exactly, so the shipped file was made with the code's formula.

The shipped file does not carry the outcome, so the check needs it from the raw data:

```
rat <- length(unique(promo$ID)) / nrow(promo)
s2 <- step2_authors %>% left_join(promo[, c("ID", "Opened")], by = "ID")
code_formula <- (s2$g_1 - s2$g_2) + s2$Wtest * rat * (s2$Opened - s2$g_1) / s2$Propensity -
  (1 - s2$Wtest) * rat * (s2$Opened - s2$g_2) / s2$Propensity
c(max_abs_difference = max(abs(code_formula - s2$Score_rat), na.rm = TRUE),
  rows_with_zero_propensity = sum(s2$Propensity == 0))
#>        max_abs_difference rows_with_zero_propensity
#>              6.984919e-10              7.000000e+00
```

Why this matters and why it does not matter much here. With the pairwise propensity $p = P(W = 1 \mid x)$ the doubly robust score divides the control residual by $1 - p$; the code divides it by p instead, which weights control residuals by the treatment propensity. The conditional mean of the code's score is no longer the treatment effect unless the outcome model is exactly right, so the code's score is not doubly robust for the control term. A second consequence is variance: the pooled propensity of an email is far smaller than its propensity within the pair, so both residual terms are divided by a small number. Section 4.4 shows what this costs in precision. We keep the paper's formula and the pairwise propensity in our own estimates.

4.2 The authors' benchmark, rerun

Rerunning the authors' first stage with their seed gives the numbers our package must match. The function below is their script with the file paths removed and the loop body kept line for line, returning the same columns as the shipped file.

```
authors_first_stage <- function(EmailOut_Analysis, n_trees = 1000, threads = 8, seed = 227) {
  set.seed(seed)
  rat <- length(unique(EmailOut_Analysis$ID)) / nrow(EmailOut_Analysis)
  Control_Email <- subset(EmailOut_Analysis, PromoID == 6)
  sample <- sample.int(n = nrow(Control_Email), size = floor(0.5 * nrow(Control_Email)), replace = FALSE)
  Control_train <- Control_Email[sample, ]
  Control_test <- Control_Email[-sample, ]
  rest_Data <- subset(EmailOut_Analysis, PromoID != 6)
  sample_rc <- sample.int(n = nrow(rest_Data), size = floor(0.5 * nrow(rest_Data)), replace = FALSE)
  rc_Treat_train <- rest_Data[sample_rc, ]
  X_P_Matrix <- rbind(rc_Treat_train[, 3:6], Control_train[, 3:6])
  W_train <- as.factor(c(rep(0, nrow(rc_Treat_train)), rep(1, nrow(Control_train))))
  p_Function_rc <- ranger(W_train ~ ., data = data.frame(X_P_Matrix, W_train), num.trees = n_trees,
                          probability = TRUE, num.threads = threads)
  out <- list()
  for (jjj in 1:5) {
    Treat_Email <- subset(EmailOut_Analysis, PromoID == jjj)
    sample_t <- sample.int(n = nrow(Treat_Email), size = floor(0.25 * nrow(Treat_Email)), replace = FALSE)
    Treat_train <- Treat_Email[sample_t, ]
    Treat_test <- Treat_Email[-sample_t, ]
    rest_rt_Data <- subset(EmailOut_Analysis, PromoID != jjj)
    sample_rt <- sample.int(n = nrow(rest_rt_Data), size = floor(0.5 * nrow(rest_rt_Data)), replace = FALSE)
    rt_Treat_train <- rest_rt_Data[sample_rt, ]
    X_P_Matrix <- rbind(Treat_train[, 3:6], rt_Treat_train[, 3:6])
    W_train <- as.factor(c(rep(1, nrow(Treat_train)), rep(0, nrow(rt_Treat_train))))
    p_Function <- ranger(W_train ~ ., data = data.frame(X_P_Matrix, W_train), num.trees = n_trees,
```

```
                          probability = TRUE, num.threads = threads)
    p_treat <- predict(p_Function, data = Treat_test[, 3:6], num.threads = threads)$predictions[, 2]
    p_control <- predict(p_Function_rc, data = Control_test[, 3:6], num.threads = threads)$predictions[, 2]
    u_treat_Function <- ranger(y ~ ., data = data.frame(Treat_train[, 3:6], y = factor(Treat_train$Opened)),
                               num.trees = n_trees, probability = TRUE, num.threads = threads)
    u_control_Function <- ranger(y ~ ., data = data.frame(Control_train[, 3:6], y = factor(Control_train$Opened)),
                                 num.trees = n_trees, probability = TRUE, num.threads = threads)
    test <- rbind(Treat_test, Control_test)
    W_test <- c(rep(1, nrow(Treat_test)), rep(0, nrow(Control_test)))
    pr <- c(p_treat, p_control)
    u_1 <- predict(u_treat_Function, data = test[, 3:6], num.threads = threads)$predictions[, 2]
    u_0 <- predict(u_control_Function, data = test[, 3:6], num.threads = threads)$predictions[, 2]
    s_out <- (u_1 - u_0) + W_test * rat * ((test$Opened - u_1) / pr) - (1 - W_test) * rat * ((test$Opened - u_0) / pr)
    out[[jjj]] <- data.frame(Treat_E = jjj, Control_E = 6, Score_rat = s_out, Propensity = pr, g_1 = u_1, g_2 = u_0,
                             ID = test$ID, Cluster = test$Cluster, Wtest = W_test, Opened = test$Opened)
  }
  do.call(rbind, out)
}
step2_rerun <- authors_first_stage(promo, n_trees = n_trees, threads = n_threads)
```

The rerun reproduces the shipped file's rows and its propensities to the precision random forests allow. Table 7 matches the two by customer and email: the same customers are scored (the split is fixed by the seed), and the scores correlate at 1.000 to three decimals. Forests with 1,000 trees are not deterministic across thread schedules, so the residual differences are Monte Carlo noise of the forests, not a difference in procedure.

Table 7: The authors' first stage rerun with their seed, matched to the shipped scores by customer and email.

Email	Rows rerun	Rows shipped	Correlation of scores	Correlation of propensities
1	33940	33940	1.0000	1.0000
2	45615	45615	1.0000	1.0000
3	78305	78305	1.0000	1.0000
4	34033	34033	1.0000	1.0000
5	59911	59911	1.0000	1.0000

4.3 Our version: pairwise scores with cross-fitting

We build the same object with `dr_scores()`, pair by pair, with three deliberate differences. First, the propensity is the pairwise one, $P(\text{email } j \mid \text{email } j \text{ or baseline}, x)$, so the score uses $1 - p$ for controls as in the paper's formula. Second, both halves are used: two-fold cross-fitting scores every customer in the pair, where the authors' split scores half of the baseline and three quarters of the treated. Third, trimming at 0.01 and 0.99 drops rows, as in the paper, rather than clipping. The learners are the same probability forests with 1,000 trees; a lasso variant follows.

```
forest_learner <- function() lrn("classif.ranger", num.trees = n_trees, predict_type = "prob", num.threads = n_threads)
pair_data <- function(j) {
  d <- promo[promo$PromoID %in% c(j, baseline_email), ]
  d$W <- as.integer(d$PromoID == j)
  d
}
scores_forest <- lapply(1:5, function(j) {
  dr_scores(pair_data(j), y = "Opened", d = "W", x = covs,
            learner_p = forest_learner(), learner_mu = forest_learner(),
            folds = 2, seed = 300 + j, trim = c(0.01, 0.99))
})
names(scores_forest) <- paste0("email", 1:5)
scores_forest[[1]]
#> Doubly robust pseudo-outcomes (dr)
#>   n = 42462, treated share = 0.295, folds = 2, trimmed share = 0.311
#>   nuisances: p = classif.ranger, mu0 = classif.ranger, mu1 = classif.ranger
```

```
#>   ATE (mean of the score) = 0.16 (SE 0.0043)
#>   score quantiles (1%, 50%, 99%): -1.579,  0.229,  2.938
```

The lasso variant replaces both forests by cross-validated lasso logits on a polynomial dictionary. The paper's footnote 15 reports that lasso nuisances gave similar results; with four covariates the dictionary of squares and pairwise products is the natural flexible linear model.

```
add_dictionary <- function(d) {
  for (v in covs) d[[paste0(v, "_sq")]] <- d[[v]]^2
  pairs <- combn(covs, 2)
  for (k in seq_len(ncol(pairs))) d[[paste0(pairs[1, k], "_x_", pairs[2, k])]] <- d[[pairs[1, k]]] * d[[pairs[2, k]]]
  d
}
dict <- c(covs, paste0(covs, "_sq"), apply(combn(covs, 2), 2, paste, collapse = "_x_"))
lasso_learner <- function() lrn("classif.cv_glmnet", predict_type = "prob")
scores_lasso <- lapply(1:5, function(j) {
  dr_scores(add_dictionary(pair_data(j)), y = "Opened", d = "W", x = dict,
            learner_p = lasso_learner(), learner_mu = lasso_learner(),
            folds = 2, seed = 300 + j, trim = c(0.01, 0.99))
})
names(scores_lasso) <- paste0("email", 1:5)
```

Trimming removes what overlap cannot support. Table 8 shows the share of each pair dropped at the 0.01 and 0.99 bounds. For emails 1, 2, and 5 the dropped rows are the baseline customers the email was never sent to; for emails 3 and 4 almost nothing is dropped. The forests and the lasso predict the pairwise propensity about equally well.

Table 8: Rows dropped by trimming the pairwise propensity at 0.01 and 0.99, and the cross-fitted propensity fit, by email and learner.

Email	Rows in pair	Trimmed, forest	Trimmed, lasso	Propensity RMSE, forest	Propensity RMSE, lasso
1	61,617	31.1%	19.1%	0.371	0.424
2	77,183	24.4%	15.2%	0.383	0.474
3	120,770	0.7%	0.0%	0.444	0.491
4	61,741	0.0%	0.0%	0.407	0.404
5	96,245	22.4%	11.4%	0.448	0.438

4.4 Pairwise ATEs

The paper's Section 6.1 starts with one pair to show that the correction is material, then shows all 33 in Figure 1. Its Table 8 puts the naive open rates next to the covariate means of the two groups so the reader sees, before any model, that the email with the higher open rate went to the more engaged customers; the doubly robust ATE then reverses the naive ranking. We follow the same order.

```
pair_ex <- c(3L, 5L)
ex <- promo[promo$PromoID %in% pair_ex, ]
ex$W <- as.integer(ex$PromoID == pair_ex[1])
sc_ex <- dr_scores(ex, "Opened", "W", covs, learner_p = forest_learner(), learner_mu = forest_learner(),
                   folds = 2, seed = 7, trim = c(0.01, 0.99))
naive_ex <- diff(rev(tapply(ex$Opened, ex$W, mean)))
c(naive_difference = unname(naive_ex), dr_ate = sc_ex$ate$estimate, t_stat = sc_ex$ate$estimate / sc_ex$ate$std.error,
  trimmed_share = sc_ex$diagnostics$trimmed_share)
#> naive_difference           dr_ate           t_stat    trimmed_share
#>      -0.01189372      -0.01903020      -6.23114897       0.29122030
```

Why emails 3 and 5. They share every component difference except the subject-line length (Table 2), which is the paper's own criterion for a pair that is "akin to an A/B test": one difference in content, and a targeting difference to be removed.

The naive gap is -0.012 and the doubly robust ATE is -0.019 (t = -6.2). Email 5 went to customers

Table 9: Consumer response and pretreatment covariates of emails 3 and 5 before correcting for targeting (the layout of the paper's Table 8).

		Customers getting email 3		Customers getting email 5	
		Mean	Standard deviation	Mean	Standard deviation
Response	Open rate	0.100	0.301	0.089	0.284
Pretreatment covariate	Demographic	2.306	0.537	2.277	0.558
	Monetary	45.376	47.134	20.432	40.348
	Frequency	3.174	4.959	0.605	1.222
	Recency	554.625	714.404	1,061.313	871.856
Number of customers		71,678		47,153	

with far longer recency and fewer purchases (Table 9), so part of its lower open rate is its audience. The correction attributes the remaining gap to the email.

Table 10: Pairwise ATE of each email against the baseline at the open stage: naive difference in open rates, and doubly robust estimates from the authors' shipped scores, their script rerun, and our two versions (t-statistics in parentheses). All doubly robust estimates trim at propensities of 0.01 and 0.99.

Email	Naive difference	Authors' shipped scores	Authors' script, rerun	dr_scores(), forests	dr_scores(), lasso
1	0.1897	0.0853 (2.5)	0.0853 (2.5)	0.1600 (37.5)	0.1533 (37.5)
2	0.1104	0.1090 (8.3)	0.1090 (8.3)	0.0883 (33.5)	0.0900 (31.2)
3	0.0081	-0.0267 (-4.9)	-0.0267 (-4.9)	-0.0056 (-3.5)	0.0081 (4.7)
4	0.1525	-0.0672 (-1.5)	-0.0672 (-1.5)	0.1423 (27.9)	0.1505 (37.5)
5	-0.0038	0.0406 (9.0)	0.0406 (9.0)	0.0262 (9.2)	0.0213 (9.5)

Where the columns agree and where they do not. The shipped scores and the rerun agree to forest noise, which confirms the benchmark. The authors' columns carry t-statistics between 1.5 and 9.0; ours carry between 3.5 and 37.5 on the same pairs. The difference is the propensity in the denominator: the pooled propensity of an email is a small number for most customers, and the code divides both residual terms by it, so the authors' scores are the outcome-model contrast plus a very noisy correction. With the pairwise propensity and the paper's formula, the same forests give estimates whose standard errors are an order of magnitude smaller. Our forest and lasso versions agree within 0.014 on every email, including email 3, whose effect is close to zero and changes sign between the two learners. For emails 1, 2, and 5 the doubly robust estimates are read on the trimmed population only (Table 8), so they are effects on the customers the email could have reached, not on everyone.

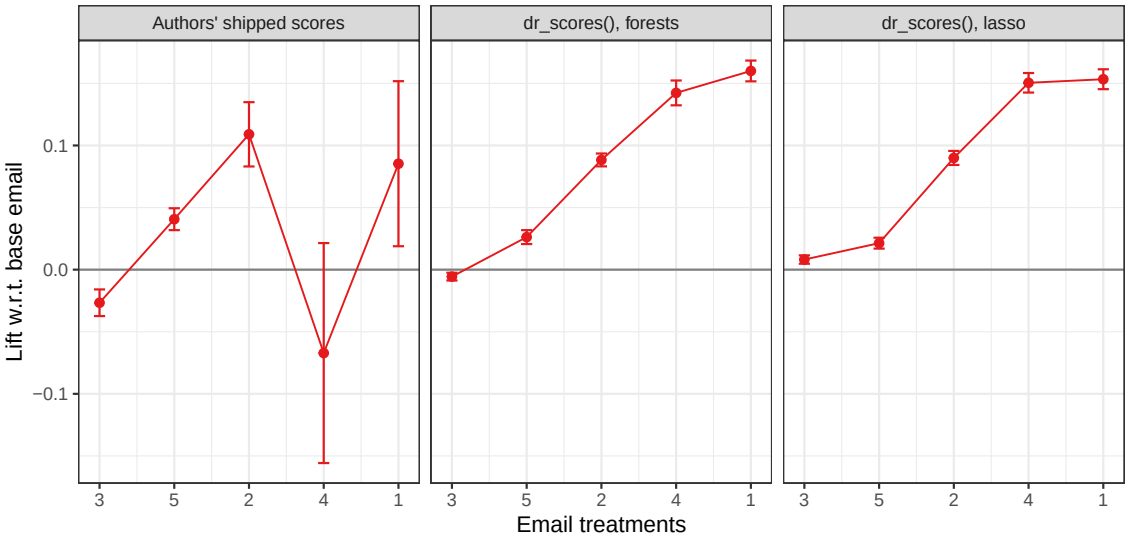

Figure 2: Average treatment effects of the five emails with respect to the baseline email at the open stage, with 95 percent error bars (the layout of the paper's Figure 1). Emails are ordered by the estimate from our forest version in every panel so the panels can be compared.

Why the figure is drawn this way. The paper sorts the 33 emails by their open-stage ATE and keeps that order in the purchase and amount figures, so a reader can see at a glance which emails move down the funnel in a different order. The connected points draw the eye to the shape of the distribution of effects, and the tight error bars make the point that the effects are precisely estimated at this sample size. Here the same order is kept across estimators instead of across outcomes.

5 Second stage: projecting the scores on the components

5.1 What the projection estimates

The second stage is a regression of the pooled scores on the component differences, and its coefficients are marginal component effects. With ψ_{ij} the score of customer i in the pair of email j with the baseline and c_j the vector of component differences of email j,

$$\beta = \arg\min_b E[(\psi_{ij} - c_j' b)^2]$$

is the best linear predictor of the conditional mean of the score, which is the pairwise treatment effect, in the components (Semenova and Chernozhukov 2021). Because the score is Neyman orthogonal, ordinary least squares with heteroskedasticity-robust standard errors is valid as if the nuisances were known. This is exactly what `cate_blp()` computes; the paper's Table 10 is its coefficient table.

Why the components are differenced from the baseline. A treatment email's score measures its effect relative to the baseline, so the regressor must measure the email's content relative to the baseline too. The intercept is then the effect of an email that shares every component with the baseline. In the paper, 33 emails identify 19 coefficients, so the projection is overidentified and the intercept is informative: a value far from zero would signal a component missing from the list.

In the replication data the projection is exactly identified, which changes how its table must be read. Five treatment emails identify five coefficients (an intercept and four components), so the regression fits the five email-level means of the scores exactly and the coefficients are a linear transformation of the five pairwise ATEs of Table 10: $\hat{\beta} = C^{-1}\widehat{\text{ATE}}$ with C the five-by-five matrix of a constant and the component differences. The check in Section 5.3 confirms it. Nothing is tested by the fit, the intercept is a free parameter,

and the standard errors are the ATE standard errors propagated through C^{-1}, which is why a component whose values barely differ across emails receives a large coefficient with a large standard error.

5.2 The authors' second stage, rerun

The authors trim, draw a stratified subsample of 25,000 scores per pair, and run one least-squares regression with HC1 standard errors. The subsample keeps the treated share of each pair; the script notes that the proprietary data were "many times larger", so the subsample is a device for the public release rather than part of the method.

```
authors_second_stage <- function(Analysis_Mat, EmailChara, seed = 226, per_pair = 25000) {
  Analysis_Mat_temp <- subset(Analysis_Mat, Propensity > .01 & Propensity < .99)
  set.seed(seed)
  temp <- NULL
  for (i in sort(unique(Analysis_Mat_temp$Treat_E))) {
    hold <- subset(Analysis_Mat_temp, Treat_E == i)
    hold_treat <- subset(hold, Wtest == 1)
    ratio <- nrow(hold_treat) / nrow(hold)
    sample_treat <- sample.int(n = nrow(hold_treat), size = floor(ratio * per_pair), replace = FALSE)
    hold_control <- subset(hold, Wtest == 0)
    sample_control <- sample.int(n = nrow(hold_control), size = per_pair - length(sample_treat), replace = FALSE)
    temp <- rbind(temp, hold_treat[sample_treat, ], hold_control[sample_control, ])
  }
  Email_Analysis <- left_join(temp, EmailChara, by = c("Treat_E" = "Email_Type"))
  m <- lm(Score_rat ~ Characters + Component1 + Component2 + Component3, data = Email_Analysis)
  list(model = m, data = Email_Analysis,
       coef = lmtest::coeftest(m, vcov = sandwich::vcovHC(m, type = "HC1")))
}
authors_ss <- authors_second_stage(step2_authors, components)
authors_ss$coef
#>
#> t test of coefficients:
#>
#>               Estimate Std. Error t value  Pr(>|t|)
#> (Intercept)  0.0028111  0.0399830  0.0703  0.943948
#> Characters  -0.0076432  0.0011809 -6.4725 9.678e-11 ***
#> Component1  -0.7655494  0.2845276 -2.6906  0.007133 **
#> Component2  -0.3103201  0.1571405 -1.9748  0.048294 *
#> Component3  -1.8194504  0.6488049 -2.8043  0.005043 **
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

5.3 Our version: `bind_scores()` and `cate_blp()`

Pooling the five score objects and projecting them is two calls. The component differences are attached to each row through the `set` column that `bind_scores()` records, so the dictionary in `cate_blp()` is the same four columns as in the authors' regression.

```
component_rows <- function(pooled) {
  idx <- match(as.integer(sub("email", "", pooled$data$set)), components$Email_Type)
  components[idx, c("Characters", "Component1", "Component2", "Component3")]
}
pooled_forest <- bind_scores(scores_forest)
pooled_lasso <- bind_scores(scores_lasso)
blp_forest <- cate_blp(pooled_forest, ~ Characters + Component1 + Component2 + Component3,
                       data = component_rows(pooled_forest), uniform = FALSE)
blp_lasso <- cate_blp(pooled_lasso, ~ Characters + Component1 + Component2 + Component3,
                      data = component_rows(pooled_lasso), uniform = FALSE)
blp_forest
#> Best linear predictor of the CATE: score ~ Characters + Component1 + Component2 + Component3
#>   n = 357126, 95% pointwise |z| = 1.96, simultaneous = 1.96
#>         term  estimate std.error  conf.low conf.high  band.low band.high
#>  (Intercept)  0.134532 0.0050014  0.124730  0.144335  0.124730  0.144335
#>   Characters -0.003184 0.0003268 -0.003825 -0.002544 -0.003825 -0.002544
#>   Component1 -0.239524 0.0295522 -0.297446 -0.181603 -0.297446 -0.181603
```

```
#>   Component2 -0.116570 0.0165974 -0.149100 -0.084040 -0.149100 -0.084040
#>   Component3 -0.240467 0.0667730 -0.371340 -0.109594 -0.371340 -0.109594
#>   Wald test of no heterogeneity: chi2(4) = 2367.74, p = 0
```

For the authors' scores we also run the full-sample projection. The shipped file has no outcome, so `cate_blp()` cannot rebuild a score object from it; the same regression with HC1 standard errors on all trimmed rows is run with `estimatr::lm_robust()`, which is the identical computation.

```
C <- cbind(1, as.matrix(components[order(components$Email_Type), c("Characters", "Component1", "Component2", "Component3")]))
ate_forest_vec <- sapply(scores_forest, function(s) s$ate$estimate)
c(max_abs_difference = max(abs(solve(C, ate_forest_vec) - blp_forest$beta)))
#> max_abs_difference
#>       6.920617e-11
```

```
shipped_full <- step2_authors %>% filter(Propensity > 0.01, Propensity < 0.99) %>%
  left_join(components, by = c("Treat_E" = "Email_Type"))
blp_shipped_full <- estimatr::lm_robust(Score_rat ~ Characters + Component1 + Component2 + Component3,
                                        data = shipped_full, se_type = "HC1")
```

Table 11: Effect of the treatment components on the open stage (the layout of the paper's Table 10). Each pair of columns shows the coefficient with significance stars and the HC1 standard error in parentheses. (1) the authors' script on their scores with the 25,000-row subsample per pair; (2) the same regression on all trimmed rows of their scores; (3) and (4) cate_blp() on our forest and lasso scores. . $p < 0.10$; * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$.

		(1) Authors' script		(2) Authors' scores, all		(3) Forests		(4) Lasso	
Type	Variables								
	(Intercept)	0.0028	(0.0400)	0.0315	(0.0344)	0.1345***	(0.0050)	0.1428***	(0.0047)
Footprint	Characters	-0.0076***	(0.0012)	-0.0067***	(0.0007)	-0.0032***	(0.0003)	-0.0013***	(0.0003)
Components	Component1	-0.7655**	(0.2845)	-0.7286**	(0.2479)	-0.2395***	(0.0296)	-0.1580***	(0.0261)
	Component2	-0.3103*	(0.1571)	-0.3015*	(0.1367)	-0.1166***	(0.0166)	-0.0731***	(0.0150)
	Component3	-1.8195**	(0.6488)	-1.6594**	(0.5651)	-0.2405***	(0.0668)	-0.0552	(0.0576)
	Observations	125,000		250,960		357,126		383,095	
	(Adjusted) R2	0.0002		0.0002		0.0062		0.0065	

Reading Table 11. Column (2) has the same coefficients as column (1) up to sampling: the subsample costs precision (its standard errors are larger by a factor near 1.1) and nothing else, which is why the authors could use it for the public release. Columns (3) and (4) transform our pairwise ATEs, and because those ATEs are precise the component coefficients are precise too; the authors' columns inherit the imprecision of their scores. Since the system is exactly identified, agreement between columns is agreement between the ATE columns of Table 10 and nothing more; the paper's overidentified Table 10, with 33 emails behind 19 coefficients, is the version in which the projection averages evidence across emails and the star pattern carries information about components.

Why the paper reports adjusted R-squared values near 0.01 without concern. The dependent variable is an individual score whose variance is dominated by the outcome residual divided by a propensity; the projection explains the conditional mean, not the individual noise. A low R-squared is expected, and the standard errors, not the fit, tell whether the component effects are learned.

6 Heterogeneous effects by engagement segment

6.1 Why two segments, and why k-means

The paper wants heterogeneity that a manager can act on, so it summarizes the fifteen covariates into two customer types before projecting. A projection on all covariates would give fifteen interaction coefficients per component, which no targeting rule uses. K-means on the standardized covariates with the elbow heuristic gives two clusters; Table 11 shows that one cluster is more recent, more frequent,

and higher spending on every variable, so the labels high engagement (HE) and low engagement (LE) need no further justification. The segments are built from pretreatment covariates only, so conditioning on them keeps the identification argument intact.

```
X_std <- scale(promo[, covs])
set.seed(1)
wss <- sapply(1:6, function(k) kmeans(X_std, centers = k, nstart = 10, iter.max = 50)$tot.withinss)
km <- kmeans(X_std, centers = 2, nstart = 10, iter.max = 50)
seg_means <- tapply(promo$Recency, km$cluster, mean)
he_cluster <- as.integer(names(which.min(seg_means)))
promo$segment <- ifelse(km$cluster == he_cluster, "High engagement", "Low engagement")
table(promo$segment)
#>
#> High engagement  Low engagement
#>          133876           87312
```

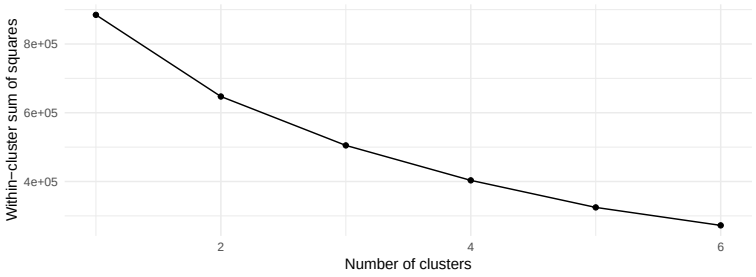

Figure 3: Within-cluster sum of squares by number of k-means clusters on the standardized covariates. The bend at two clusters is the elbow the paper uses.

Table 12: Summary statistics of the pretreatment covariates in the two k-means segments (the layout of the paper's Table 11).

		High engagement		Low engagement	
Recipient characteristic	Pretreatment covariate	Mean	Standard deviation	Mean	Standard deviation
Demographic	Demographic	2.27	0.52	2.32	0.56
Monetary	Monetary	70.65	42.38	2.74	8.66
Frequency	Frequency	4.91	5.59	0.19	0.49
Recency	Recency	229.23	178.62	1,137.59	923.45
Number of customers		133,876		87,312	

6.2 Component effects by segment

Tables 12 and 13 of the paper are Table 10 run twice, once per segment, and the reader is meant to compare the same coefficient across the two. The paper's finding is that most components work better for engaged customers, with a few reversals at the purchase-amount stage. The segment enters the projection as a sample split rather than as an interaction so that every coefficient, the intercept included, is segment specific.

```
attach_segment <- function(pooled) {
  pooled$data$segment <- promo$segment[match(pooled$data$ID, promo$ID)]
  pooled
}
pooled_forest <- attach_segment(pooled_forest)
subset_scores <- function(pooled, keep) {
  out <- pooled
  out$score <- pooled$score[keep]; out$y <- pooled$y[keep]; out$d <- pooled$d[keep]
```

```
  out$nuisance <- pooled$nuisance[keep, ]; out$residuals <- pooled$residuals[keep, ]
  out$data <- pooled$data[keep, ]; out$fold_id <- pooled$fold_id[keep]; out$n <- sum(keep)
  out$ate <- list(estimate = mean(out$score), std.error = sd(out$score) / sqrt(out$n))
  out
}
blp_by_segment <- lapply(c("High engagement", "Low engagement"), function(s) {
  sub <- subset_scores(pooled_forest, pooled_forest$data$segment == s)
  cate_blp(sub, ~ Characters + Component1 + Component2 + Component3, data = component_rows(sub), uniform = FALSE)
})
names(blp_by_segment) <- c("High engagement", "Low engagement")
shipped_full$segment <- promo$segment[match(shipped_full$ID, promo$ID)]
shipped_by_segment <- lapply(c("High engagement", "Low engagement"), function(s) {
  estimatr::lm_robust(Score_rat ~ Characters + Component1 + Component2 + Component3,
                      data = shipped_full[shipped_full$segment == s, ], se_type = "HC1")
})
```

Table 13: Effect of the treatment components on the open stage by engagement segment (the layout of the paper's Tables 12 and 13, side by side). Coefficient with significance stars and HC1 standard error in parentheses; (1)-(2) cate_blp() on our forest scores, (3)-(4) the same regression on the authors' trimmed scores. . $p < 0.10$; * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$.

		(1) HE, forests		(2) LE, forests		(3) HE, authors		(4) LE, authors	
Type	Variables								
Footprint Components	(Intercept)	0.1336***	(0.0079)	0.1152***	(0.0216)	-0.0730	(0.0452)	0.2463***	(0.0420)
	Characters	-0.0037***	(0.0008)	-0.0019***	(0.0003)	-0.0108***	(0.0010)	-0.0026**	(0.0010)
	Component1	-0.2015***	(0.0345)	-0.1930.	(0.1105)	-0.3864	(0.3126)	-1.6084***	(0.3769)
	Component2	-0.0975***	(0.0188)	-0.0752	(0.0667)	-0.1427	(0.1733)	-0.6789***	(0.2039)
	Component3	-0.1267	(0.0823)	-0.2091	(0.2248)	-1.0190	(0.7070)	-3.3934***	(0.8898)
	Observations	226,409		130,717		153,841		97,119	
	(Adjusted) R2	0.0070		0.0036		0.0001		0.0009	

Reading Table 13. Within each segment the projection is again exactly identified, so each column is a transformation of that segment's five GATEs (Section 6.3). For 3 of the four components the coefficient is larger in absolute value for the engaged segment, the pattern the paper reports for its real components. Segment-specific coefficients are less precise than the pooled ones because each regression uses about half of the rows, which is why the paper reports the pooled Table 10 first and the segment tables as a refinement.

6.3 Group average effects of each email

Before projecting on components, the paper's first projection class is the GATE of each pairwise contrast on customer groups. `cate_gate()` gives the doubly robust GATE of every email by segment with intervals that are simultaneous across the two segments and a test that the segments differ. This is the object the off-policy exercise needs.

```
gate_by_email <- lapply(1:5, function(j) {
  s <- scores_forest[[j]]
  seg <- promo$segment[match(s$data$ID, promo$ID)]
  cate_gate(s, seg, n_boot = 499, seed = 10 + j)
})
gate_tab <- bind_rows(lapply(1:5, function(j) gate_by_email[[j]]$table %>% mutate(Email = j, p_equal = gate_by_email[[j]]$test_e
```

7 Off-policy evaluation

7.1 Why the paper ends with a counterfactual

The paper closes by turning the estimates into money because a component effect is only useful if it changes a targeting decision. Its Section 6.5 takes the two clearance emails of Table 8, notes that the firm sent the more generous deal to the less engaged customers, and asks what revenue would be under

Table 14: Doubly robust group average treatment effect of each email against the baseline, by engagement segment, with pointwise 95 percent intervals and the p-value of the test that the two segments share one effect.

Email	Segment	n	GATE	Std. error	95% CI	p (equal)
1	High engagement	37,904	0.1635	0.0040	[0.1556, 0.1714]	0.131
	Low engagement	4,558	0.1307	0.0214	[0.0887, 0.1726]	
2	High engagement	52,731	0.0876	0.0026	[0.0824, 0.0928]	0.516
	Low engagement	5,608	0.0953	0.0116	[0.0727, 0.1179]	
3	High engagement	75,359	-0.0136	0.0021	[-0.0177, -0.0094]	0.000
	Low engagement	44,562	0.0078	0.0024	[0.0032, 0.0124]	
4	High engagement	39,617	0.1584	0.0070	[0.1447, 0.1720]	
	Low engagement	22,123	0.1136	0.0068	[0.1003, 0.1269]	
5	High engagement	20,798	0.0239	0.0082	[0.0078, 0.0400]	0.707
	Low engagement	53,866	0.0271	0.0024	[0.0225, 0.0318]	

the swap. Doubly robust scores make this cheap: the value of any rule that assigns emails from pretreatment variables is an average of scores, so the same first-stage objects that produced the ATEs evaluate a new policy without new data (Dudík, Langford, and Li 2011). The paper reports an 11 percent revenue lift.

7.2 The pair of emails 3 and 5 with our scores

Within the pair, a policy is a rule that sends email 3 rather than email 5, and `policy_value()` prices it against sending either email to everyone. The scores of Section 4.4 treat email 3 as the treatment, so a rule's value is its gain in opens over sending email 5 to all, and the gain over sending email 3 to all is reported by `baseline = "all"`.

```
sc_ex$data$segment <- promo$segment[match(sc_ex$data$ID, promo$ID)]
gate_ex <- cate_gate(sc_ex, "segment", n_boot = 499, seed = 3)
gate_ex$table[, c("group", "n", "estimate", "std.error", "conf.low", "conf.high")]
#>              group     n    estimate   std.error    conf.low    conf.high
#> 1 High engagement 21981 -0.02488859 0.009940471 -0.04437156 -0.005405629
#> 2  Low engagement 62244 -0.01696135 0.002180603 -0.02123525 -0.012687447
rules <- list(
  send_3_to_high_engagement = as.numeric(sc_ex$data$segment == "High engagement"),
  send_3_to_low_engagement = as.numeric(sc_ex$data$segment == "Low engagement"),
  send_3_to_everyone = rep(1, sc_ex$n)
)
ope_vs_5 <- policy_value(sc_ex, rules, baseline = "none")
ope_vs_3 <- policy_value(sc_ex, rules, baseline = "all")
```

Table 15: Off-policy evaluation of rules that choose between emails 3 and 5 by segment, on the trimmed pair sample: gain in the open rate per customer over the two uniform policies, with standard errors from the doubly robust scores.

Comparison	Rule	Share sent email 3	Gain in open rate	Std. error	95% CI
Gain over sending email 5 to everyone	send 3 to high engagement	0.261	-0.0065	0.0026	[-0.0116, -0.0014]
	send 3 to low engagement	0.739	-0.0125	0.0016	[-0.0157, -0.0094]
	send 3 to everyone	1.000	-0.0190	0.0031	[-0.0250, -0.0130]
Gain over sending email 3 to everyone	send 3 to high engagement	0.261	0.0125	0.0016	[0.0094, 0.0157]
	send 3 to low engagement	0.739	0.0065	0.0026	[0.0014, 0.0116]
	send 3 to everyone	1.000	0.0000	0.0000	[0.0000, 0.0000]

```
sc_ex$data$segment_he <- as.numeric(sc_ex$data$segment == "High engagement")
pol_ex <- policy_learn(sc_ex, x = "segment_he", method = "tree", depth = 1, seed = 1)
```

What the table says. The segment GATEs give the direction. Email 5 beats email 3 in both segments

(-0.025 and -0.017 for the engaged and the disengaged), so no segment rule beats sending email 5 to everyone: every gain in the first block is negative. The paper's pair had a reversal across segments, which is why a swap paid there. Here the off-policy evaluation says that the firm should not swap, which is the point of evaluating a rule before deploying it.

Why the paper uses separate samples for estimation and prediction. A rule chosen on the same scores that evaluate it is chosen to look good on them. `policy_learn()` holds out half of the rows before searching and reports the held-out value; the learned depth-one rule below sends email 3 to no one, and its held-out gain over sending email 3 to everyone is the honest version of the last block of the table.

```
pol_ex$value
#>       sample     n share_treated value_vs_none se_vs_none value_vs_all
#> 1 estimation 42113             0             0          0   0.02164181
#> 2    holdout 42112             0             0          0   0.01641853
#>     se_vs_all
#> 1 0.004718001
#> 2 0.003879322
```

7.3 An email whose effect reverses by segment

Email 3 against the baseline is the pair with a reversal. Its GATE is -0.014 for the engaged segment and 0.008 for the disengaged (Table 14), so the engaged should keep the baseline and the disengaged should receive email 3. The rule that sends email 3 only to the disengaged is priced below against sending it to everyone and to no one, and a depth-one tree on the segment indicator learns the same rule on held-out rows.

```
s3 <- scores_forest[[3]]
s3$data$segment_he <- as.numeric(promo$segment[match(s3$data$ID, promo$ID)] == "High engagement")
rules3 <- list(send_3_to_disengaged_only = 1 - s3$data$segment_he,
               send_3_to_engaged_only = s3$data$segment_he,
               send_3_to_everyone = rep(1, s3$n))
policy_value(s3, rules3, baseline = "all")
#>                      policy share_treated     estimate    std.error
#> 1 send_3_to_disengaged_only     0.3715946  0.008523678 0.0013291122
#> 2    send_3_to_engaged_only     0.6284054 -0.002902551 0.0008779071
#> 3       send_3_to_everyone     1.0000000  0.000000000 0.0000000000
#>       conf.low    conf.high diff_vs_first     diff_se
#> 1  0.005918667  0.011128690   0.000000000 0.000000000
#> 2 -0.004623217 -0.001181885  -0.011426230 0.001592748
#> 3  0.000000000  0.000000000  -0.008523678 0.001329112
pol3 <- policy_learn(s3, x = "segment_he", method = "tree", depth = 1, seed = 2)
pol3
#> Treatment policy (tree, depth 1, exhaustive)
#>   cost = 0, share treated = 0.372
#> if segment_he <= 0.5:
#>   action = 1 (treat)
#> else:
#>   action = 0 (do not treat)
#>   value over no treatment / over treating everyone:
#>      sample     n share_treated value_vs_none se_vs_none value_vs_all se_vs_all
#>  estimation 59961        0.3708     7.089e-06   0.001244     0.006058  0.001879
#>     holdout 59960        0.3724     5.798e-03   0.001239     0.010989  0.001880
```

The gain is small in absolute terms but precisely estimated. Sending email 3 only to the disengaged, rather than to everyone, raises the open rate by the first row's estimate per customer, and the held-out value of the learned rule confirms it. Scaled to the number of recipients, this is the analogue of the paper's revenue comparison between its factual and counterfactual policies.

8 What the replication teaches about the workflow

One score object carries the whole analysis. The pairwise ATEs, the component projection, the segment projections, the GATEs, and the policy values are all averages or regressions of the same doubly

robust scores. Building the scores once, with cross-fitted nuisances and a recorded trimming rule, and then reusing them is the design of the paper and of the package.

Sample splitting is the ingredient that makes forests admissible. The authors train on one half and score the other; two-fold cross-fitting does the same for both halves. Either way, no customer's outcome enters the nuisance model that scores it. That is what removes the overfitting bias of Section 04, and the price is a smaller effective training sample for each nuisance.

Multi-treatment propensities are a modelling choice with consequences for the trimmed population. The paper estimates each email's propensity against the pooled population and stabilizes it; the pairwise propensity is the alternative. In the paper's data the two agree because overlap holds. In the synthetic data they disagree for the three emails whose targeting was deterministic, and the disagreement shows up as different trimmed populations and different ATEs. Balance tables are read before estimation for this reason.

The code is the estimator. The script divides both residuals by the treatment email's pooled propensity while the paper's formula uses the pairwise control propensity for the control term. The shipped file was reproduced from its own columns with the code's formula, which is how the difference was found, and its cost was visible in the ATE table: the authors' scores carry standard errors an order of magnitude larger than the same forests give with the paper's formula. Replication means matching the script, then judging the script against the paper.

Subsampling in the projection is a release device, not part of the method. The 25,000-row subsample changes the standard errors and nothing else; the full-sample projection with HC1 errors is `cate_blp()`, and it also provides simultaneous bands that the paper does not report.

Low R-squared values in the projection are expected. The dependent variable is an individual score with a propensity in its denominator; the regression explains its conditional mean, and the standard errors carry the inference.

Segments and policies are where the estimates become decisions. Two clusters on pretreatment variables are enough to show that component effects differ by engagement, and the off-policy value of a segment rule is a mean of scores with a standard error. The held-out value of a learned rule is the honest number to report.

9 References

- Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., and Robins, J. (2018). Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1), C1-C68.
- Crump, R. K., Hotz, V. J., Imbens, G. W., and Mitnik, O. A. (2009). Dealing with limited overlap in estimation of average treatment effects. *Biometrika*, 96(1), 187-199.
- Dudík, M., Langford, J., and Li, L. (2011). Doubly robust policy evaluation and learning. *Proceedings of the 28th International Conference on Machine Learning*, 1097-1104.
- Ellickson, P. B., Kar, W., and Reeder, J. C. (2023). Estimating marketing component effects: Double machine learning from targeted digital promotions. *Marketing Science*, 42(4), 704-728.
- Hernán, M. A. and Robins, J. M. (2006). Estimating causal effects from epidemiological data. *Journal of Epidemiology and Community Health*, 60(7), 578-586.
- McCaffrey, D. F., Griffin, B. A., Almirall, D., Slaughter, M. E., Ramchand, R., and Burgette, L. F. (2013). A tutorial on propensity score estimation for multiple treatments using generalized boosted models. *Statistics in Medicine*, 32(19), 3388-3414.
- Semenova, V. and Chernozhukov, V. (2021). Debiased machine learning of conditional average treatment effects and other causal functions. *The Econometrics Journal*, 24(2), 264-289.